

DNS 中继服务器

实验报告

课题名称DNS 中继服务器 (DNS Relay Server)

张恒基 (2024210926)

小组成员尹浩铭 (2024210910)

林旭东 (2024210915)

班 级2024211301

完成日期2026 年 5 月

北京邮电大学

Beijing University of Posts and Telecommunications

摘 要

本报告围绕 DNS 中继服务器课程设计课题，说明从需求分析、系统设计、关键实现到测试验证的完整过程。我们在 Linux/WSL 环境下使用 C 语言实现了一个运行于 UDP 53 端口的 DNS 中继程序：根据本地配置文件 `dnsrelay.txt`，对域名查询执行本地拦截、本地解析或上游转发，并将符合 RFC 1035 规范的响应返回客户端。

实现过程中，我们重点处理了 DNS 报文格式与网络字节序、域名长度前缀编码与指针压缩、Transaction ID 映射、`select()` 事件驱动主循环、上游超时与错误响应等问题。测试表明，系统在本地解析、域名拦截、公网中继及 QTYPE 边界场景下均能给出符合协议预期的结果。

关键词：DNS • UDP • RFC 1035 • 中继服务器 • select • 网络字节序

目 录

目录

- 1 需求分析 1
 - 1.1 背景与课题意义 1
 - 1.2 一次查询在本系统中的生命周期 2
 - 1.3 设计目标 2
 - 1.4 功能需求 3
 - 1.5 非功能需求 3
 - 1.6 协议基础 3
- 2 系统设计 6
 - 2.1 总体架构 6
 - 2.2 模块划分 7
 - 2.3 主循环流程 8
 - 2.4 三种业务分支的设计思路 9
 - 2.5 ID 映射表设计 9
 - 2.6 小组分工 10
- 3 关键实现说明 11
 - 3.1 网络字节序与报文首部 11
 - 3.2 域名编解码与安全 11
 - 3.3 配置表的加载与匹配 12
 - 3.4 本地响应的构造原则 12
 - 3.5 上游中继与超时处理 13
- 4 测试与结果分析 14
 - 4.1 测试环境 14
 - 4.2 测试用例设计 15
 - 4.3 实测结果与分析 15

4.4 测试结论	16
5 总结	17
5.1 开发过程中遇到的主要问题	17
5.2 收获	17
5.3 不足与改进方向	18
6 参考文献	19
7 附录	20
7.1 编译与运行	20
7.2 测试命令	20
7.3 项目文件结构	21

1 需求分析

1.1 背景与课题意义

1.1.1 DNS 在互联网中的角色

当用户在浏览器中输入 `www.bupt.edu.cn` 时，计算机并不会直接连接这个字符串。操作系统会先把域名交给 DNS 解析服务，询问对应的 IPv4 地址；得到答案后，才向该 IP 发起 TCP 连接。DNS 因此处于应用层与传输层之间的关键位置，负责把人类可读的域名映射为机器可用的 IP 地址。

1.1.2 本课题要做什么

本课程设计不要求实现完整的递归 DNS 服务器，而是实现一个轻量级 DNS 中继 (Relay) 服务器：

- 1. 在本机 UDP 53 端口监听客户端 DNS 查询；
- 2. 查阅本地策略表，决定拦截、本地应答或转发上游；
- 3. 构造或转发 DNS 响应，返回给发起查询的客户端。

这种「中继 + 本地策略」模型与校园网、企业网中的 DNS 过滤、内网域名映射场景十分接近，具有明确的工程背景。

1.1.3 与生产级 DNS 的差异

能力	生产级递归 DNS	本课题实现
域名数据	完整递归查询链	仅本地 txt 配置表
缓存	TTL 感知缓存	无
传输	UDP + TCP	仅 UDP
记录类型	A/AAAA/MX/CNAME 等	主要处理 A 记录
并发	多进程/异步 I/O	单线程 + select

表 1 本课题实现与生产级 DNS 的能力对比（刻意简化）

我们有意聚焦 RFC 1035 报文、UDP Socket、`select()` 多路复用和本地策略表，把课程知识点落到一份可编译、可演示、可测试的完整程序上。

1.2 一次查询在本系统中的生命周期

理解「一条查询从进入到离开」的全过程，有助于把握整体设计：

步骤	阶段	说明
1	接收	客户端向本机 53 端口发送 UDP 查询，含域名、QTYPE、Transaction ID
2	调度	<code>select()</code> 感知可读， <code>recvfrom</code> 读入报文；长度 < 12 则丢弃
3	解析	协议模块提取 Question；非法报文返回 FORMERR
4	查表	在 <code>dnsrelay.txt</code> 中查找域名，决定拦截/本地/中继
5	应答	构造本地响应，或转发上游后还原 ID 回传
6	完成	客户端收到响应，一次解析结束

表 2 DNS 查询在本系统中的处理生命周期

设计要点

每一步都有明确的失败路径：格式错误、上游超时、映射表满等场景均不会 silent failure，而是返回对应 RCODE，保证客户端行为可预期。

1.3 设计目标

本项目的设计围绕以下四个核心目标展开：

第一，严格遵循 RFC 1035 协议规范。DNS 报文具有精确的二进制格式，所有多字节字段必须使用网络字节序传输，域名采用长度前缀标签编码。偏离规范的实现会导致与标准 DNS 客户端无法互操作。

第二，采用事件驱动模型避免 CPU 忙等待。使用 POSIX `select()` 阻塞等待 socket 可读，超时设为 10ms，既保证响应及时，又避免空转占满 CPU。

第三，实现严格的模块化分离。协议编解码、配置管理、ID 映射、主控逻辑分文件实现，便于测试与后续替换（如升级为 epoll 异步模型）。

第四，支持多客户端并发查询。虽然上游中继当前为同步阻塞，但 `select()` 本身非阻塞，多个客户端请求可在循环中依次处理；ID 映射表确保响应正确路由。

1.4 功能需求

模式	触发条件	对外行为	典型场景
本地拦截	配置 IP 为 0.0.0.0	RCODE=3 (NXDOMAIN)	屏蔽广告/恶意域名
本地解析	配置非零 IPv4	返回 A 记录，TTL=300s	内网域名、课程测试域
上游中继	域名不在配置表	转发上游，还原 ID 后返回	访问 baidu.com 等公网域

表 3 三种核心功能模式

配置文件 `参考资料/dnsrelay.txt` 每行格式为 `IP 域名`，支持 `#` 注释。程序启动时一次性加载；加载失败则退化为纯中继模式。

配置示例：

```
# 本地解析
123.127.134.10 bupt
202.108.33.89 sina

# 拦截
0.0.0.0 008.cn
```

1.5 非功能需求

- 1. 协议合规：报文格式、字节序、QTYPE 一致性符合 RFC 1035；
- 2. CPU 友好：主循环用 `select()` 带 10ms 超时，避免忙等待；
- 3. 模块化：协议、配置、ID 映射、主控分文件实现；
- 4. 可测试性：支持 `nslookup` / `dig` / 自研脚本分别验证三种模式。

1.6 协议基础

DNS 报文经 UDP 传输，单条上限 512 字节，由 12 字节首部与 Question/Answer/Authority/Additional 四个区段组成。

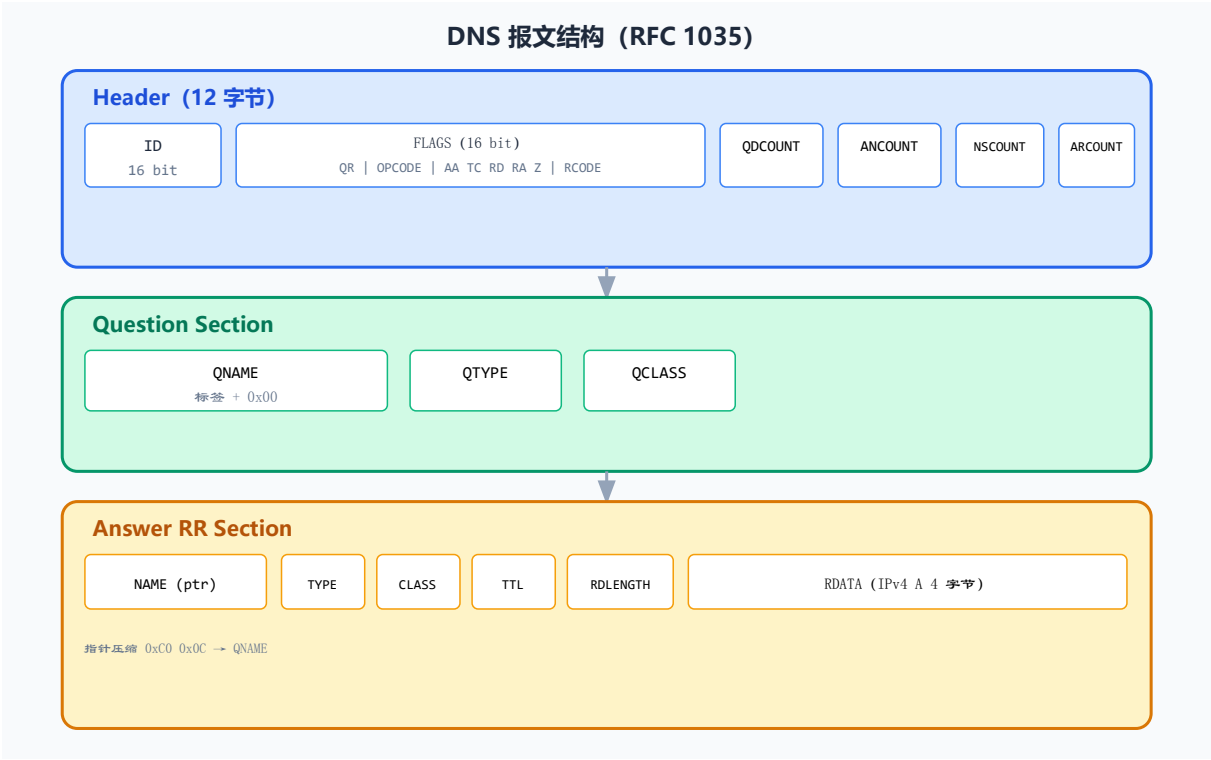


图 1 DNS 报文结构示意图

域名编码：采用长度前缀标签。例如 `www.bupt.edu.cn` 编码为 `\x03www\x04bupt\x03edu\x02cn\x00`——每标签前一字节表长度，以 `\x00` 结束。单标签最长 63 字节，完整域名最长 255 字节。

指针压缩：Answer 段可引用 Question 中已出现的域名，用两字节指针（高 2 位为 `11`）表示报文内偏移，例如 `0xc00c` 指向偏移 12 处 QNAME 起始。

字节序：多字节字段在线缆上使用大端序；x86 主机为小端序，读写报文时必须通过 `htons` / `ntohs` 等函数显式转换。

值	名称	本系统用途
0	NOERROR	本地解析成功；非 A 类型本地命中返回空应答
1	FORMERR	报文格式错误（长度不足、QDCOUNT≠1、域名解码失败）
2	SERVFAIL	上游超时（3 秒）或 sendto/recvfrom 失败
3	NXDOMAIN	配置 IP 为 0.0.0.0，执行本地拦截

表 4 本系统使用的 DNS 响应码（RCODE）

2 系统设计

2.1 总体架构

DNS 中继服务器采用单进程、单线程的事件驱动架构。服务启动后完成 socket 创建、端口绑定、配置加载，随后进入以 `select()` 为核心的无限主循环。

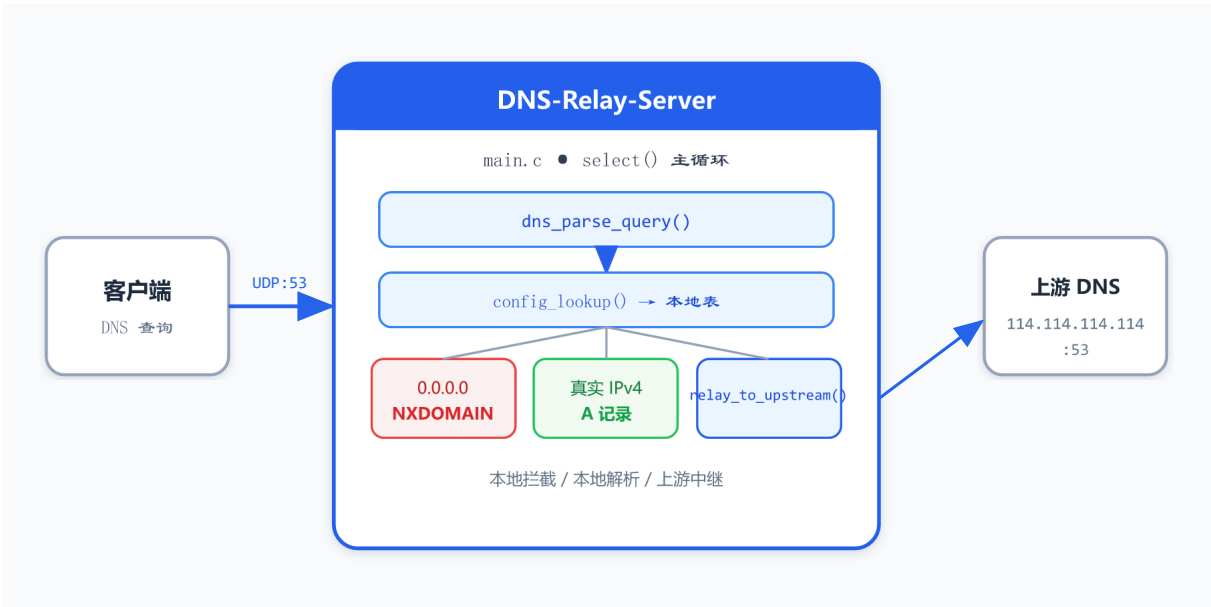


图 2 系统总体架构

架构选型理由：

1. 单进程单线程：DNS 查询报文短、逻辑简单，课程并发量下单线程足够，且避免 ID 映射表的竞态访问；
2. 事件驱动：`select()` 在无流量时不占满 CPU，有流量时及时响应；
3. 模块分离：协议细节与业务调度解耦，便于单独调试与升级。

系统启动时序：`socket()` → `setsockopt(SO_REUSEADDR)` → `bind(0.0.0.0:53)` → `config_load()` → 进入主循环。绑定地址和端口可通过环境变量 `DNS_RELAY_BIND`、`DNS_RELAY_PORT` 覆盖。

2.2 模块划分

模块	源文件	职责说明
协议层	<code>dns_protocol.*</code>	报文解析、域名编解码、A 记录与错误响应构造
配置层	<code>config.*</code>	加载/查询 <code>dnsrelay.txt</code> ，大小写不敏感域名匹配
ID 映射	<code>id_map.*</code>	记录中继时的 ID 与客户端地址，超时清理
主控层	<code>main.c</code>	Socket 生命周期、select 循环、三路分支调度与上游中继

表 5 模块划分与职责

模块间仅通过头文件声明的接口交互。主控层调用「解析查询」「查配置」「构造响应」等高层接口，不直接操作报文字节偏移，降低耦合度。编译使用 GNU Make，`wildcard` 自动发现 `src/*.c` 源文件。

2.3 主循环流程

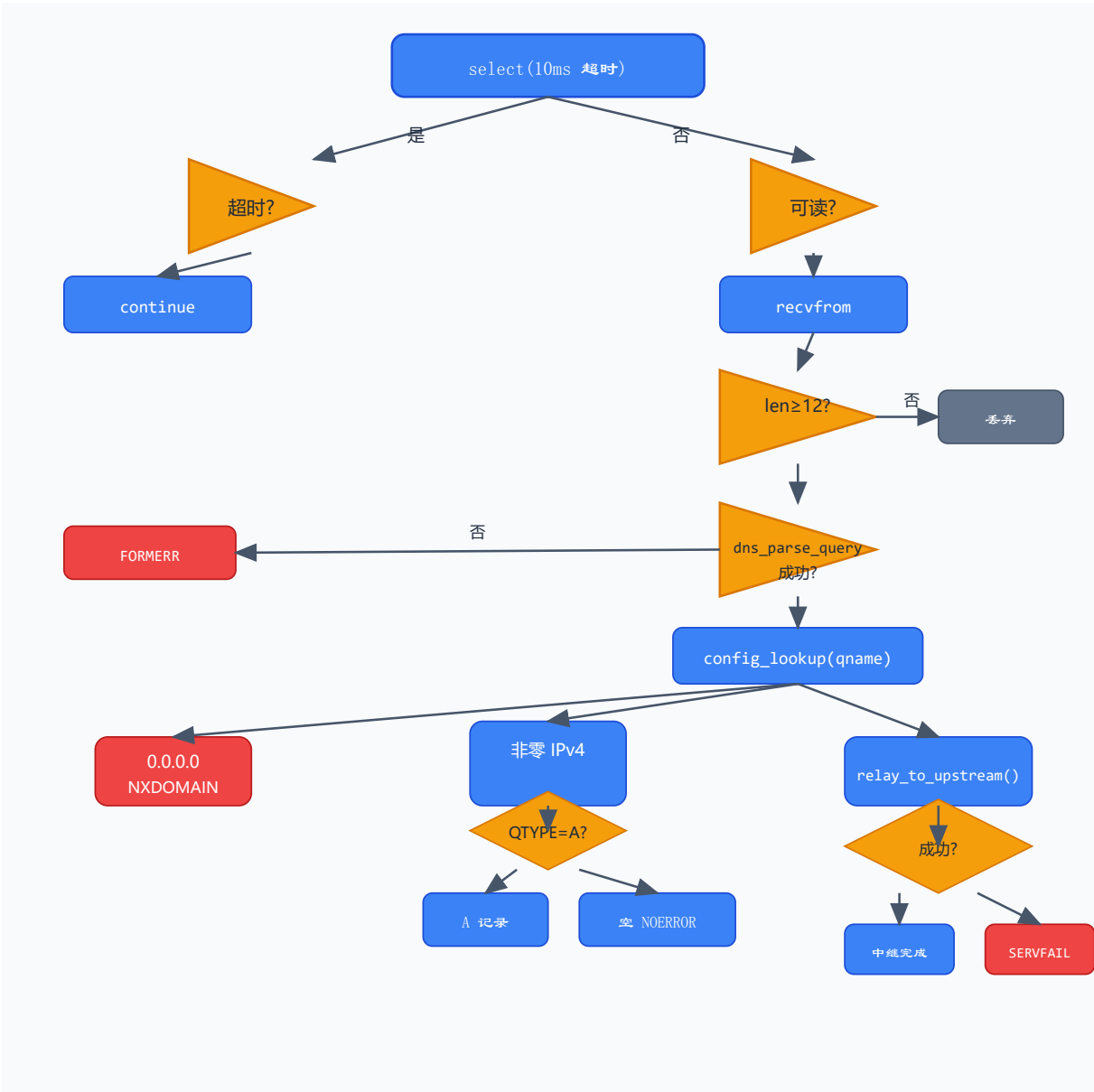


图 3 主循环处理流程

主循环各阶段说明：

1. `select` 等待：注册监听 `socket`，10ms 超时。被信号中断（`EINTR`）则重试；超时则进入下一轮。
2. 接收报文：`recvfrom` 读入 UDP 数据；失败则记录错误并继续服务，不中断进程。
3. 长度校验：少于 12 字节（DNS 首部最小长度）则静默丢弃，可能是扫描流量或误发包。
4. 超时清理：每次收包前清理 ID 映射表中超过 5 秒的过期记录。
5. 查询解析：提取域名与 `QTYPE`；解析失败返回 `FORMERR`。
6. 配置查表：命中 `0.0.0.0` → `NXDOMAIN`；命中非零 IP → 按 `QTYPE` 分支；未命中 → 上游中继。
7. 中继失败：上游超时或发送失败时返回 `SERVFAIL`，避免客户端 `indefinite` 等待。

为何用 `select` 而非忙等待？若主循环在无数据时不断调用 `recvfrom`，CPU 占用会接近 100%。

`select()` 让进程在无流量时休眠，有流量时被唤醒，是网络服务器编程中的经典模式。

2.4 三种业务分支的设计思路

2.4.1 本地拦截

配置 `0.0.0.0 域名` 表示该域名不应被解析。服务器不转发上游，直接构造 NXDOMAIN 响应。

客户端（浏览器、`nslookup`）会理解为「域名不存在」，达到屏蔽效果。从协议角度看，这是对 RCODE 语义的正确使用。

2.4.2 本地解析

配置真实 IP 时，本机扮演该域名的应答者：复制原查询的 Header 与 Question，在 Answer 段填入 A 记录，NAME 字段用指针压缩引用 Question 中的域名，避免重复编码。

重要边界：QTYPE 一致性

仅当 QTYPE=A 时才返回 A 记录。对 MX、AAAA 等查询，即使域名在配置表中，也应返回空 NOERROR (ANCOUNT=0)，否则 Answer 的 TYPE 与 Query 的 QTYPE 不一致，部分客户端会异常。这是测试阶段发现并修复的关键细节。

2.4.3 上游中继

未配置域名需转发至公网 DNS（默认 `114.114.114.114`）。

Transaction ID 冲突 是中继的核心难点：多个客户端可能使用相同 ID 发起查询。转发时必须为本机向上游发出的每条查询分配新 ID，并记录「新 ID ↔ 原始 ID ↔ 客户端地址」；收到上游响应后，再将 ID 还原为客户端原始值后回传。

当前实现为同步中继：发完查询后阻塞等待响应（最长 3 秒），此期间主循环无法接收新报文。

对课程规模可接受；面向更高并发应改为 `epoll` 异步模型。

2.5 ID 映射表设计

映射表固定容量 1024 条，采用环形槽位分配：维护一个全局写指针，依次寻找下一个空闲槽位插入记录；槽位满时线性扫描找空位。

每条记录保存：客户端原始 ID、转发用新 ID、客户端 IP 与端口、创建时间戳。

1. 老化：主循环每次收包调用 `clear_timeout_records(now, 5)`，清理 5 秒前的记录；

2. 表满处理：`add_record` 失败时先 `clear_timeout_records(now, 0)` 强制清空全部记录，再重试一次；仍失败则中继返回 `-1`，主循环向客户端返回 `SERVFAIL`。

当前同步中继模型下，响应回传主要依赖调用栈中的 `client_addr`；映射表主要用于记录与超时回收。若升级为异步模型，映射表将成为响应路由的核心数据结构。

2.6 小组分工

成员	学号	主要负责
张恒基	2024210926	主控逻辑、select 主循环、上游中继、集成测试
尹浩铭	2024210910	DNS 协议层：报文解析、域名编解码、响应构造、字节序
林旭东	2024210915	配置模块、ID 映射表、测试脚本、报告整理

表 6 小组成员与分工

分工按模块划分；联调阶段三人共同完成 QTYPE 边界、上游超时、并发等场景的排查与修复。

3 关键实现说明

本章从「遇到什么问题 → 为何这样设计 → 实际效果」说明实现要点，不罗列 API 清单。

3.1 网络字节序与报文首部

问题：x86 为小端序，DNS 线缆格式为大端序。若直接将结构体 memcpy 到报文，FLAGS、QDCOUNT 等字段会被颠倒，dig 或 nslookup 会报格式错误。

方案：内存中用主机序操作结构体，写入报文缓冲前统一转换为网络序，读出后做反向转换。

FLAGS 字段包含多个按位划分的子字段，位域布局因编译器而异，因此封装专用转换函数屏蔽平台差异。

首部字段	宽度	说明
ID	16 bit	事务标识；中继转发时需替换并在响应中还原
FLAGS	16 bit	含 QR(查询/响应)、RCODE(结果码) 等
QDCOUNT	16 bit	Question 段记录数；本系统要求为 1
ANCOUNT	16 bit	Answer 段记录数；本地 A 记录时为 1
NSCOUNT / ARCOUNT	各 16 bit	本系统本地应答时均为 0

表 7 DNS 首部字段与本项目的关系

3.2 域名编解码与安全

域名编解码是 DNS 报文处理中最易出错的环节。

编码：将 bupt 转为 \x04bupt\x00。需校验：标签长度不超过 63 字节、完整域名不超过 255 字节、不允许连续点号产生零长度标签。

解码：顺序读取各标签；遇到压缩指针（首字节高 2 位为 11）时，跳转到报文指定偏移继续读取。

风险	可能后果	对策
循环指针	解码陷入死循环	跳转次数上限 10
非法偏移	越界读取、段错误	每次访问前校验 offset < 512
畸形标签	解析失败或误读	严格校验 label_len 与报文 边界

表 8 域名解码中的风险与应对

正常 DNS 报文中指针跳转通常不超过 2~3 次，10 次上限提供了充足安全裕度。

3.3 配置表的加载与匹配

配置表在启动时一次性读入内存，最多 4096 条。逐行读取文件，跳过空行和 # 注释，解析 IP 与域名两个字段，用 inet_pton 校验 IPv4 格式——无效行输出警告后跳过，不终止整个加载过程。

查表时使用大小写不敏感比较。DNS 协议规定域名不区分大小写，BUPT 与 bupt 语义等价；若用普通 strcmp，会导致配置了记录却查不到的情况。

3.4 本地响应的构造原则

无论拦截、本地解析还是错误响应，均遵循同一原则：保留原查询的 Header 与 Question，只修改响应相关字段。

场景	QR	RCODE	ANCOUNT	含义
本地 A 记录	1	0	1	成功返回 IPv4 地址
本地拦截	1	3	0	域名不存在（被策略屏蔽）
格式错误	1	1	0	报文无法解析
上游失败	1	2	0	转发超时或网络错误
非 A 本地命中	1	0	0	域名存在但无此类型记录

表 9 各类响应的标志位设置

本地 A 记录应答时，Answer 段的 NAME 使用压缩指针 0xC00C 引用偏移 12 处 QNAME，TYPE=A、CLASS=IN、TTL=300、RDATA 为 4 字节 IPv4 地址。

3.5 上游中继与超时处理

上游通信使用临时 UDP Socket（避免干扰主监听 socket 的绑定状态），设置 3 秒接收超时。
中继完整链路：分配新 ID → 写入映射表 → 修改查询报文 ID → 发送至 114.114.114.114 → 等待响应 → 还原 ID → 回传客户端 → 关闭临时 socket。
早期版本中，上游不可达时程序 silent failure，客户端（如 nslookup）会 indefinite 等待。现改为：中继失败时主动返回 SERVFAIL，客户端可立即感知「解析服务暂时不可用」。

4 测试与结果分析

4.1 测试环境

项目	配置
操作系统	WSL2 Ubuntu 24.04 (Windows 主机运行 Linux 工具链)
编译器	gcc 14, <code>-Wall -Wextra -std=c11 -g</code>
运行权限	<code>sudo ./dnsrelay</code> (53 端口需 root); 测试可用 5353 端口
验证工具	<code>nslookup</code> 、 <code>dig</code> 、 <code>scripts/dns_query.py</code>
上游 DNS	<code>114.114.114.114</code> (国内公共 DNS, 响应快)

表 10 测试环境

选择 WSL 而非 Windows 原生编译，是因为项目依赖 `arpa/inet.h`、`sys/select.h` 等 POSIX 接口，Windows 不提供这些头文件。WSL 提供完整 Linux 用户态，GCC + Make 开箱即用。

编译与运行：

```
make clean && make

sudo ./dnsrelay

# WSL 测试端口 (正式验收仍用 53)

DNS_RELAY_BIND=127.0.0.1 DNS_RELAY_PORT=5353 ./dnsrelay
```

4.2 测试用例设计

编号	命令	预期	验证点
1	nslookup bupt 127.0.0.1	123.127.134.10	本地解析
2	nslookup sina 127.0.0.1	202.108.33.89	第二条本地记录
3	nslookup 008.cn 127.0.0.1	NXDOMAIN	本地拦截
4	nslookup baidu.com 127.0.0.1	公网真实 IP	上游中继
5	nslookup -type=mx bupt 127.0.0.1	空 NOERROR	QTYPE 分支
6	双终端同时查 baidu.com	均成功	基本并发

表 11 测试用例设计

用例 1~4 覆盖三种核心模式；用例 5 验证 QTYPE 边界；用例 6 验证多客户端下 ID 映射与中继互不干扰。

4.3 实测结果与分析

测试在 WSL2 Ubuntu 24.04 下进行，`make clean && make` 零警告通过。因本机 53 端口被 `systemd-resolved` 占用，测试阶段使用 5353 端口；正式验收与课程演示仍使用默认 53 端口。

用例 1 — bupt 本地解析

脚本输出：`qname=bupt, rcode=0, ancourt=1, len=38`。

RCODE=0 表示 NOERROR；ANCOUNT=1 表示 Answer 段有一条 A 记录；报文 38 字节符合「12 字节首部 + Question + 16 字节 Answer RR」的预期长度。与配置 `123.127.134.10 bupt` 完全一致，本地解析功能正常。

用例 3 — 008.cn 本地拦截

脚本输出：`qname=008.cn, rcode=3, ancourt=0, len=24`。

RCODE=3 即 NXDOMAIN; ANCOUNT=0 表示无 Answer 段; 24 字节比 A 记录响应更短。客户端收到后会认为域名不存在, 拦截策略生效。

用例 4 — baidu.com 上游中继

脚本输出: `qname=baidu.com, rcode=0, ancourt=4, len=91`。

域名未在本地表中, 程序转发至 114.114.114.114; 上游返回 4 条 A 记录 (baidu.com 通常有多接入 IP)。Transaction ID 在转发前后保持一致, ID 还原逻辑正确。

用例 5 — MX 查询 bupt

对 bupt 发起 MX 类型查询时, 服务器返回 NOERROR 但 Answer 段为空, 未错误返回 A 记录, 验证了 QTYPE 分支判断的正确性。

用例 6 — 并发测试

两终端同时查询 baidu.com, 均收到有效响应。主循环通过 select 依次处理两个请求, 各自分配独立 Transaction ID, 证明基本并发能力满足课程要求。

完整终端输出见 `docs/test-output.txt`。提交 PDF 时, 建议将 `nslookup` 或 `dns_query.py` 的终端截图附在附录中。

4.4 测试结论

综合以上用例, 本系统实现了课程要求的全部功能: 本地拦截、本地解析、上游中继均符合预期; 边界场景 (非 A 查询、上游超时、畸形报文) 有明确错误响应, 程序运行稳定。

5 总结

5.1 开发过程中遇到的主要问题

问题	现象	原因	解决
FLAGS 解析	dig 报格式错	位域与字节序	封装转换函数
指针死循环	程序卡死	循环指针	跳转上限 10
指针越界	段错误	非法偏移	offset 校验
MX 误返 A	类型不匹配	未判 QTYPE	非 A 返回空 NOERROR
上游挂起	客户端久等	无超时/无回包	3s 超时 + SERVFAIL
Windows 编译	缺头文件	非 POSIX	改用 WSL2

表 12 主要问题与解决方案

这些问题大多在边界测试与工具联调阶段才暴露，说明网络协议程序不能只验证 happy path，必须考虑异常输入、超时、并发等真实场景。

5.2 收获

通过本项目，我们对 DNS 协议的理解从概念层面深入到了字节层面——亲手构造 Header、域名编码、指针压缩、Answer RR 组包，理解了 RFC 1035 的设计动机（例如压缩指针用 14 位偏移是为节省空间）。

1. 掌握了网络字节序与主机字节序的区别，以及 Socket 编程中「在报文边界显式转换」的可复用模式；
2. 理解了 `select()` 事件驱动与忙等待的本质差异——10ms 超时使 CPU 从近 100% 降至近乎 0；
3. 实践了模块化 C 工程组织：头文件接口、Makefile 自动发现源文件、分模块联调。

同时也体会到：中继服务器逻辑虽不如完整递归 DNS 复杂，但在 Transaction ID 管理、QTYPE 分支、错误响应等方面仍有大量细节需要注意。

5.3 不足与改进方向

1. 同步中继阻塞主循环：等待上游期间无法接收新报文。可引入 epoll 将上游 socket 纳入事件循环，实现真正异步中继。
2. ID 映射表与同步模型未完全协同：当前响应回传依赖调用栈；异步化后映射表需成为路由核心。
3. 无 TTL 缓存：相同域名重复查询每次都打上游。生产级中继通常加入 TTL 感知缓存。
4. 仅支持 IPv4 A 记录：未处理 AAAA、CNAME，也未支持 EDNS0 与 TCP fallback。

这些局限不影响课程要求完成，但为后续扩展留下了清晰方向。

6 参考文献

1. Mockapetris P. RFC 1035: Domain Names - Implementation and Specification. IETF, 1987.
2. Mockapetris P. RFC 1034: Domain Names - Concepts and Facilities. IETF, 1987.
3. 谢希仁. 计算机网络（第 8 版）. 北京：电子工业出版社，2021.
4. Stevens W. R., Fenner B., Rudoff A. M. UNIX Network Programming, Volume 1 (3rd Edition). Addison-Wesley, 2004.

7 附录

7.1 编译与运行

环境要求：POSIX 兼容系统（Linux/WSL2）、GCC（C11）、GNU Make、UDP 53 端口可用、上游 DNS 可达。

```
make clean && make

sudo ./dnsrelay

# 非特权端口测试

DNS_RELAY_BIND=127.0.0.1 DNS_RELAY_PORT=5353 ./dnsrelay
```

7.2 测试命令

```
sudo sh scripts/test_dns.sh

nslookup bupt 127.0.0.1

nslookup 008.cn 127.0.0.1

nslookup baidu.com 127.0.0.1

nslookup -type=mx bupt 127.0.0.1

python3 scripts/dns_query.py 127.0.0.1 5353 bupt 008.cn baidu.com
```

7.3 项目文件结构

路径	说明
include/	头文件：协议、配置、ID 映射接口
src/	实现：main、dns_protocol、config、 id_map
参考资料/ dnsrelay.txt	域名—IP 配置表（208 条）
diagrams/	架构图、流程图、报文结构图（SVG）
scripts/	测试脚本 dns_query.py、test_dns.sh
Makefile	编译规则
实验报告.typ	本报告 Typst 源文件

表 13 项目目录结构